Strategy Pattern

↔ Behavioural pattern..

Problem

"You have multiple ways to perform the same task"

Ex:

- Payment → UPI / CARD / WALLET.
- Sorting → Quick sort / Merge sort.

Most beginners write:

```
if type == 'A'
    do_this()
elif type == 'B'
    do_that()
```

⇒ This many, uncalable & hard to maintain

Strategy Solution:-

- ☆ Seperate each behaviour into its own class
- ☆ Make them interchangable at runtime

" Encapsulate a family of algorithms & make them interchangable "

Intent:-

define an own algorithm ↑ binding together ↑

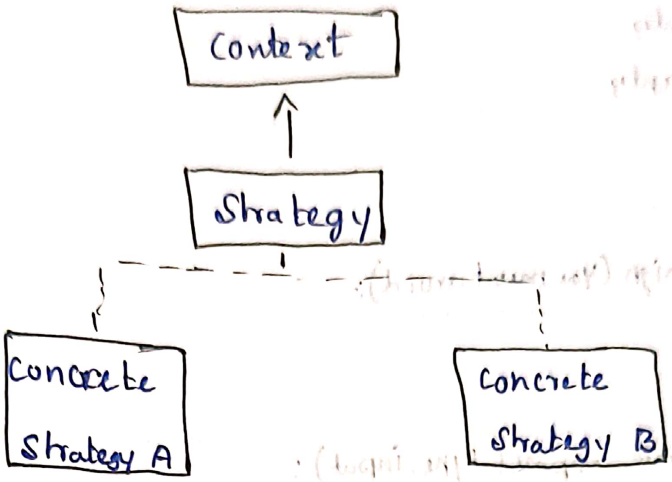
" Define a family of algorithm, encapsulate each one, and make them interchangable "

↓

according functionality we can use them..

Scanned with OKEN Scanner

☆ By also let the algorithm vary independently from clients that uses it..



understanding the flow:- {3 main parts}

① Strategy interface:-

Define a common method

strategy

└ Execute ();

② Concrete Strategies:-

Different implementation

- Formal Response strategy
- Casual response strategy
- Technical response strategy..

③ Context (Main system)

uses a strategy

which is the most likely useful...

Scanned with OKEN Scanner

Real world Example:

AI response generator.

- Formal reply
- Casual reply
- Technical reply

↓

BAD System Design (You must avoid):

```
def generate_response(type, input):
    if type == "formal":
        return "Good evening " + Input
    elif type == "casual":
        return "Hey bro!" + input
    elif type == "technical":
        return "According to system analysis" + Input.
```

Problems:

- Adding New type → Modify function x

violates open-closed principle

Hard to scale..

Scanned with OKEN Scanner

Good Design (Strategy)

Step - I ⇒ strategy interface:

```
class ResponseStrategy:
    def generate(self, input_text):
        pass
```

Step - II ⇒ Concrete strategies:

```
class FormalStrategy(ResponseStrategy):
    def generate(self, input_text):
        return "Good evening: " + Input_text

class CasualStrategy(ResponseStrategy):
    def generate(self, input_text):
        return "Hey!" + input_text

class TechnicalStrategy(ResponseStrategy):
    def generate(self, input_text):
        return "From a technical perspective. " +
```

Step - III ⇒ Context (Main AI System):

```
class AIResponseSystem:
    def __init__(self, strategy: ResponseStrategy):
        self.strategy = strategy
    def set_strategy(self, strategy : ResponseSt
        self.strategy = strategy
```

Scanned with OKEN Scanner

```
def generate_response(self, input_text):
    return self.strategy.generate(input_text)
```

Step-IV

Using It:

```
ai = AIResponseSystem(FormalStrategy())
print(ai.generate_response("Your request is processed"))

ai.set_strategy(CasualStrategy())
print(ai.generate_response("done"))

ai.set_strategy(TechnicalStrategy())
print(ai.generate_response("task executed"))
```

Scanned with OKEN Scanner